5

SESIÓN 5

Arquitectura orientada a eventos: mensajería vs. streaming · contratos asíncronos · gobierno de esquemas

Semana 5 · 3 horas (180 min) · 6:00 – 9:00 p. m. · Caso PetroAndes

5 Objetivos y preparación

TRES OBJETIVOS, TRES BLOQUES DE PRÁCTICA

1. Diferenciar mensajería tradicional (colas) de streaming de eventos, y elegir según el escenario
2. Documentar la API asíncrona con AsyncAPI 3.0 y encapsular los eventos con CloudEvents
3. Explicar el schema registry y las reglas de compatibilidad evolutiva de esquemas

LECTURAS PREVIAS (≤ 90 MIN)

- Bellemare, Building Event-Driven Microservices: fundamentos y comunicación event-driven
- AsyncAPI 3.0: tutorial oficial (channels, operations, messages)
- CloudEvents (CNCF): atributos obligatorios del sobre

ACTIVIDAD DETONANTE (LMS)

Para detectar válvulas ilícitas, ¿por qué un log de eventos es preferible a una cola punto-a-punto? Piense en re-procesamiento y en múltiples consumidores.

5 Agenda de la sesión (180 min)

Bloque	Min	Actividad	Objetivo
0	5	Apertura y encuadre de la sesión	—
1	40	Teoría activa: mensajería tradicional vs. streaming de eventos	1
2	25	Práctica A: elegir el estilo, la clave de partición y la retención	1
R	**15**	**Receso**	—
3	35	Teoría: contratos asíncronos — CloudEvents y AsyncAPI 3.0	2
4	30	Práctica B: escribir el contrato telemetria-ductos	2
5	25	Schema registry y compatibilidad evolutiva + ejercicio de rupture	3
6	5	Cierre, ticket de salida y lanzamiento del laboratorio	—

Cada objetivo tiene su bloque de teoría y su bloque de práctica sobre PetroAndes. La implementación se hace en el laboratorio guiado de la semana.

5 Bloque 1 — El vocabulario primero

40 min de teoría activa · se abre con las respuestas a la actividad detonante

OBJETIVO 1

EVENTO · COMANDO · CONSULTA

- **Evento**
  Algo ocurrió. Hecho inmutable, en pasado. No sabe quién lo escucha.
- **Comando**
  Haz esto. Tiene destinatario y puede ser rechazado.
- **Consulta**
  Dime esto. Espera respuesta, no cambia el estado.

El error más común: AlertaGenerada es un evento; EnviarAlerta es un comando disfrazado de evento.

TRES ESTILOS DE EVENTO

Notificación ligera
Solo el identificador. Bajo acoplamiento, pero el consumidor debe devolverse a preguntar.

Event-carried state transfer
El evento carga el estado. Consumidor autónomo, a costa de payloads mayores.

Event sourcing
El log es la fuente de verdad. Se retoma en la sesión 6: reconstruir el balance ante la ANH.

5 Bloque 1 — Cola tradicional vs. log distribuido

OBJETIVO 1

COLA TRADICIONAL

- El mensaje se consume y desaparece; el broker lleva la cuenta
- Un consumidor por mensaje, o competencia entre consumidores
- Excelente para repartir trabajo entre ejecutores
- Malo para releer la historia: lo consumido ya no está
- RabbitMQ, SQS, colas JMS clásicas

LOG DISTRIBUIDO

- El mensaje se retiene; cada consumidor lleva su propio offset
- Varios consumidores independientes leen lo mismo sin duplicar publicación
- Cualquiera puede rebobinar y reprocesar
- Vocabulario: topic, partición, offset, consumer group, retención
- Kafka, Redpanda, Pulsar (híbrido), Kinesis

En PetroAndes: la conciliación mensual reprocesa seis meses de telemetría sin pedirle nada al productor.

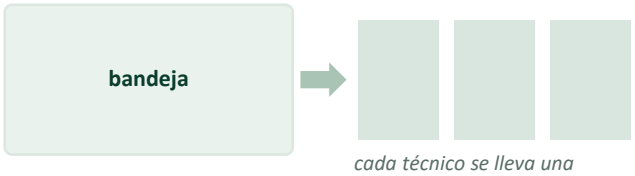
El orden solo existe dentro de una partición.

Garantías de entrega: at-most-once · at-least-once · exactly-once semántico. La opción honesta en la mayoría de casos: at-least-once con consumidor idempotente.

5 Cola o log: la canasta de órdenes y el libro de bitácora

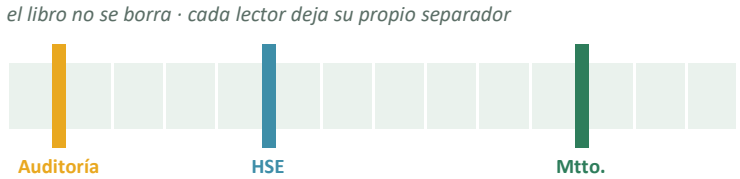
Misma estación de bombeo, dos formas de repartir la información

COLA · LA CANASTA DE ÓRDENES



- El papel sale de la bandeja: quien lo toma se lo lleva
- Perfecto para repartir trabajo, nadie hace lo mismo dos veces
- El auditor llega el mes siguiente y la bandeja está vacía

LOG · EL LIBRO DE BITÁCORA



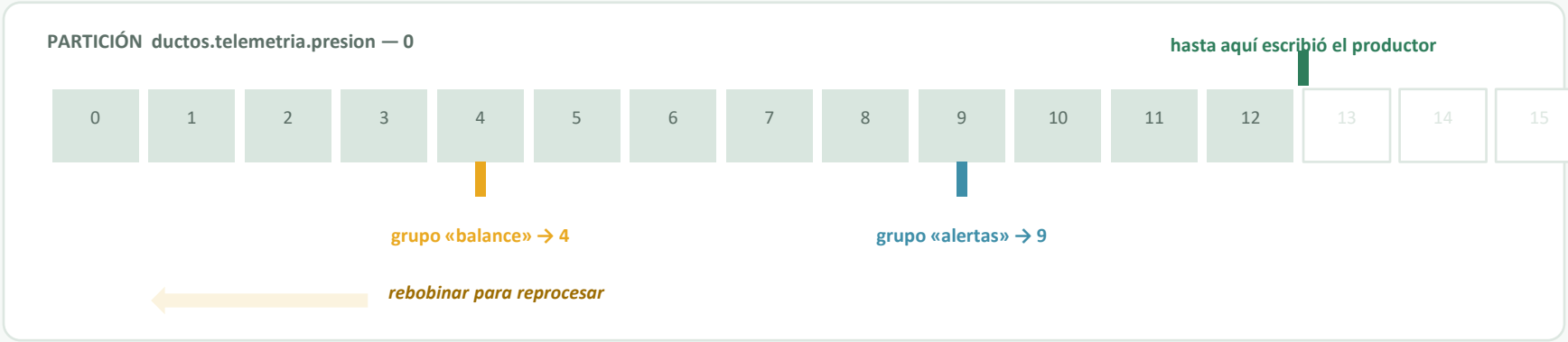
- Nadie estorba a nadie: cada área avanza a su velocidad
- Releer es devolver el separador unas páginas
- El auditor abre en la primera página tres meses después

DEL LIBRO AL VOCABULARIO TÉCNICO

topic el libro	**offset** el separador	**consumer group** el área que comparte separador	**retención** años en el archivo	**partición** un libro por tramo

5 El offset: el separador de cada lector

Un entero que crece de uno en uno y marca la posición del mensaje dentro de UNA partición



DOS OFFSETS QUE NO HAY QUE CONFUNDIR

Offset del log:
hasta dónde escribió el productor. Lo controla el broker.

Offset comprometido:
hasta dónde dice cada consumer group que ya procesó. Lo controla el consumidor.

QUÉ HABILITA QUE EL SEPARADOR SEA SUYO

- Cada grupo avanza a su propia velocidad sobre el mismo topic
- Rebobinar para reprocesar; o saltar al final para ver solo lo nuevo
- Tras una caída, se retoma desde el último offset comprometido

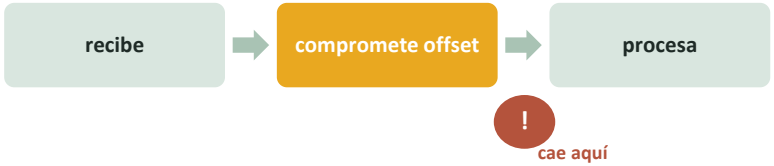
El offset es local a la partición: el 4.312 del tramo Ayacucho–Coveñas y el 4.312 de Galán–Bucaramanga no tienen relación alguna.

5 Garantías de entrega: cuándo se compromete el offset

La misma caída del consumidor, en tres diseños distintos

AT-MOST-ONCE

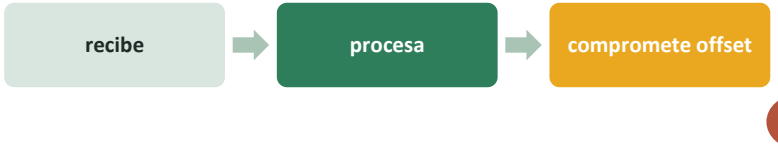
Cero duplicados · posible PÉRDIDA



Al reiniciar arranca después del mensaje que nunca terminó.

AT-LEAST-ONCE

Cero pérdida · posibles DUPLICADOS



Al reiniciar vuelve a procesar el mismo mensaje.

EXACTLY-ONCE

semántico, no de entrega

recibe · procesa · compromete — en una sola transacción

El mensaje sí puede viajar dos veces; lo que se garantiza es el efecto.

Vale solo dentro de la transacción: al escribir en SAP o llamar a un servicio externo, vuelve a ser at-least-once.

Recomendación del curso: at-least-once + consumidor idempotente (llave natural del evento y upsert, nunca acumulación ciega).

5 Streaming: el log no solo transporta, también procesa

Con estado y por ventanas de tiempo — es lo que convierte el batch nocturno en una respuesta de minutos

OBJETIVO 1

SIN ESTADO vs. CON ESTADO

Sin estado

Filtrar, transformar, enrutar. Cada mensaje se resuelve solo, sin recordar nada.

Con estado

Contar, sumar, promediar, unir dos flujos, detectar un patrón sostenido. El procesador debe recordar lo que ya vio — y ese estado hay que ubicarlo, respaldarlo y recuperarlo tras una caída.

VENTANAS: CÓMO SE LE PONE BORDE AL FLUJO

Tumbling *contiguas, sin solape*



balance por hora: 00–01, 01–02…

Sliding *se solapan: avanzan de a poco*



presión promedio de los últimos 10 min, recalculada cada 2

Session *agrupan por actividad, con pausas*



un bombeo, desde que arranca hasta que para

Eventos tardíos: la lectura del campo remoto llega 20 min después. ¿Se reabre la ventana?

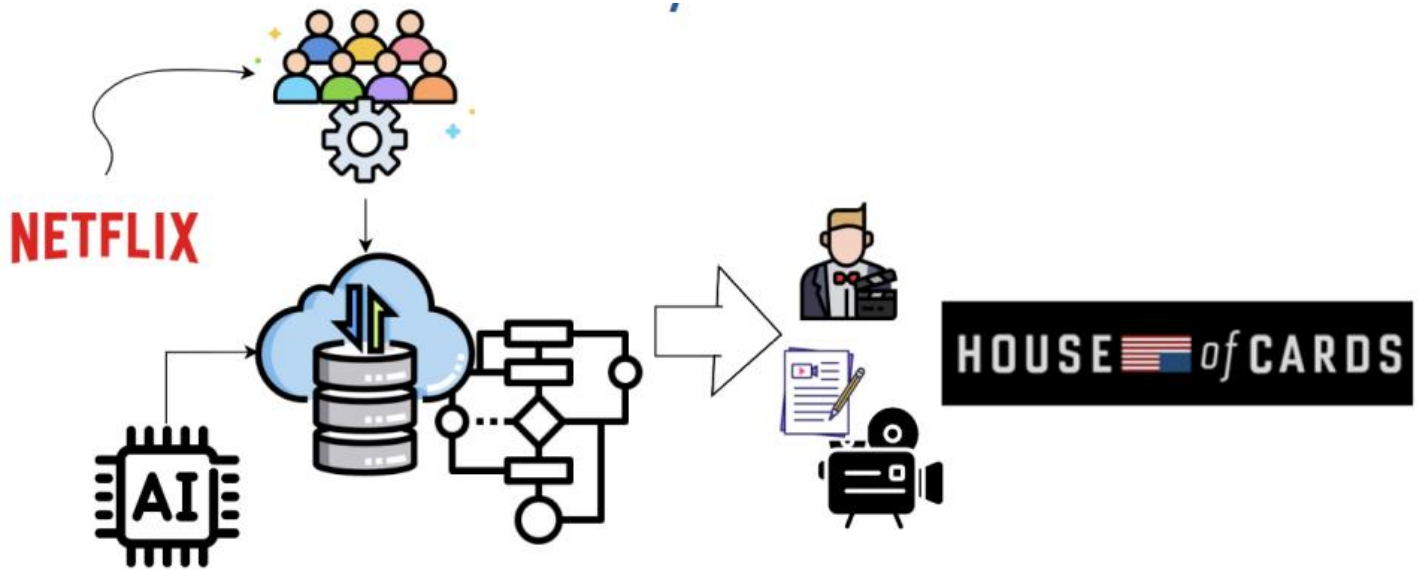
Kafka Streams, Flink, ksqlDB — el procesamiento vive fuera del broker.

En PetroAndes: el balance del tramo deja de ser un batch nocturno y pasa a calcularse por ventana horaria en el flujo; la anomalía se detecta mientras el producto todavía se está perdiendo.

5 Streaming: el log no solo transporta, también procesa

Con estado y por ventanas de tiempo — es lo que convierte el batch nocturno en una respuesta de minutos

OBJETIVO 2



Edge computing: Mejorar la experiencia de usuario con baja latencia, mayor calidad de video, personalización avanzada, mayor privacidad

Inteligencia artificial: Incorporación de AI para mejorar las recomendaciones y la personalización.

Realidad virtual y aumentada: Alta interactividad con eventos y decisiones en tiempo real, personalización dinámica y análisis de comportamiento.

5 Event mesh: cuando un solo broker no alcanza

Varios brokers interconectados entre el campo, la DMZ industrial y la nube — el puente hacia la sesión 6

OBJETIVO 1

NUBE Y CORPORATIVO · Purdue 4–5

balance, analítica, agentes, lakehouse

broker central

publica hacia arriba

DMZ INDUSTRIAL · Purdue 3.5

punto único de cruce, gobernado

puente / réplica

nunca al revés

CAMPO Y PLANTA · Purdue 0–2

SCADA, sensores, historiador local

broker en el borde

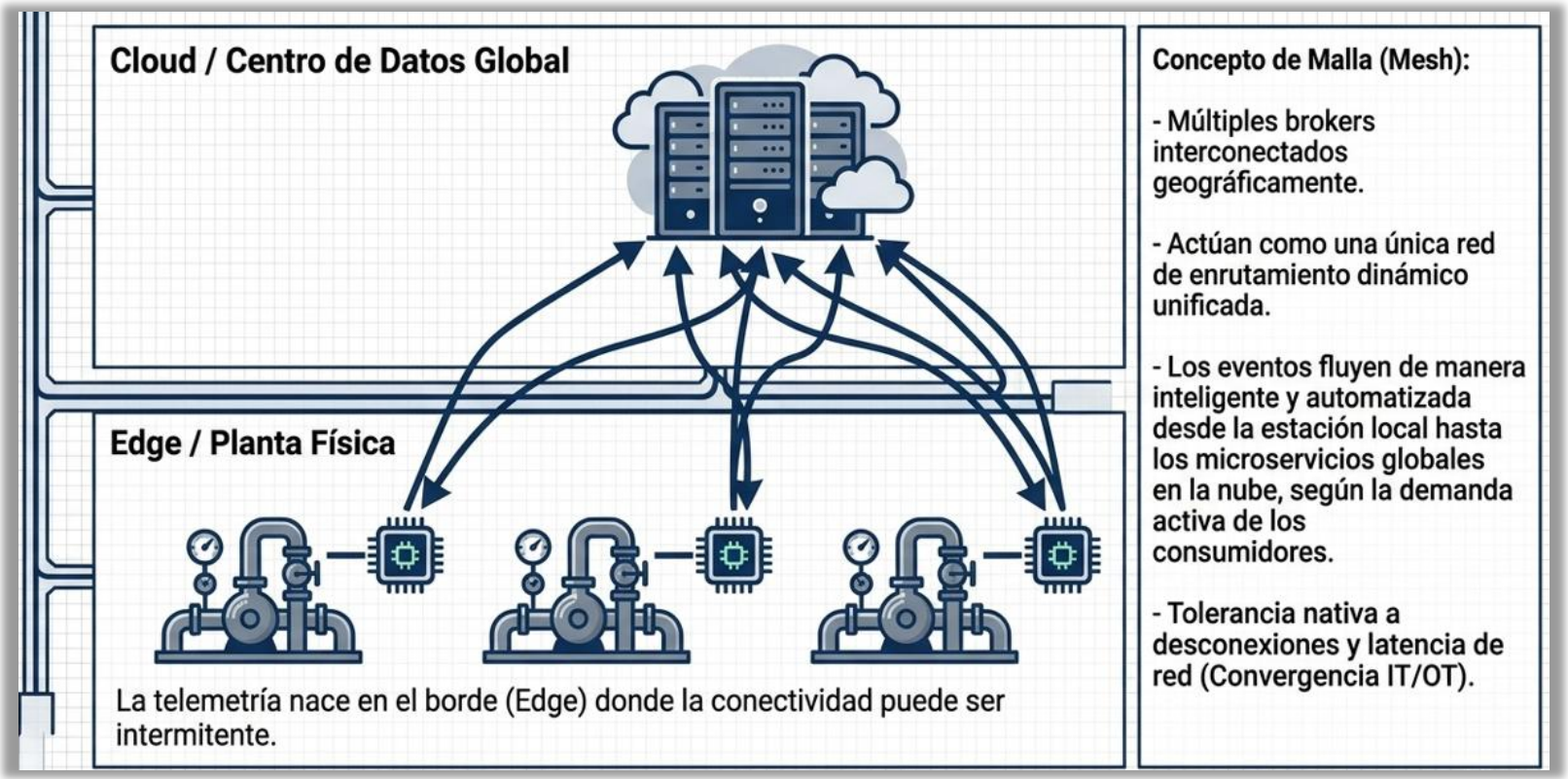
QUÉ RESUELVE EL MESH

- Buffering en el borde cuando el enlace satelital cae: cero pérdida de eventos
- Filtrar y agregar antes de subir: no todo el tag de un historiador merece viajar a la nube
- Replicación selectiva de topics, no del clúster completo
- Un solo punto gobernado de cruce, con esquemas y permisos compartidos

La separación IT/OT no es excusa para no integrar: el mesh es la forma de respetarla y aun así publicar el dato.
Se retoma en la sesión 6 junto con el Modelo Purdue, la DMZ industrial e IEC 62443.

5 Event mesh: cuando un solo broker no alcanza

Varios brokers interconectados entre el campo, la DMZ industrial y la nube — el puente hacia la sesión 6



5 Práctica A — Tres interfaces, tres decisiones

25 min · en grupos · se entrega media página al final del bloque

OBJETIVO 1

¿COLA O LOG? JUSTIFIQUE CON LOS CRITERIOS

1. **Telemetría de presión y caudal del poliducto**
   consumida por balance, alertas y analítica

2. **Órdenes de despacho a los cargaderos**
   un ejecutor por orden, sin releer

3. **Nominaciones mensuales de los remitentes**
   volumen bajo, pico el primer día del mes

PARA LA QUE RESUELVAN CON LOG

- Clave de partición elegida y qué orden garantiza
- Política de retención y su justificación de negocio: ¿cuánta historia exige re-liquidar un balance ante la ANH?
- Qué pasa si un consumidor se cae 10 minutos, y qué pasa si se cae 10 días

Punto a dejar sentado en la puesta en común (8 min finales)
Particionar por estación da paralelismo, pero rompe el orden dentro del tramo — que es justo lo que el balance necesita.

5 Bloque 3 — CloudEvents: estandarizar el sobre

12 min · el argumento del modelo canónico, aplicado a los metadatos

OBJETIVO 2

ATRIBUTOS DEL SOBRE

OBLIGATORIOS

- **id**
- **source**
- **specversion**
- **type**

ÚTILES EN LA PRÁCTICA

- **time**
- **subject**
- **datacontenttype**
- **dataschema**

dataschema es lo que amarra el evento con el registry del bloque 5. Omitirlo es el error más frecuente.

POR QUÉ EL SOBRE Y NO SOLO EL CONTENIDO

Gateway, mesh, trazador y dead-letter pueden enrutar, deduplicar y auditar sin entender el payload.

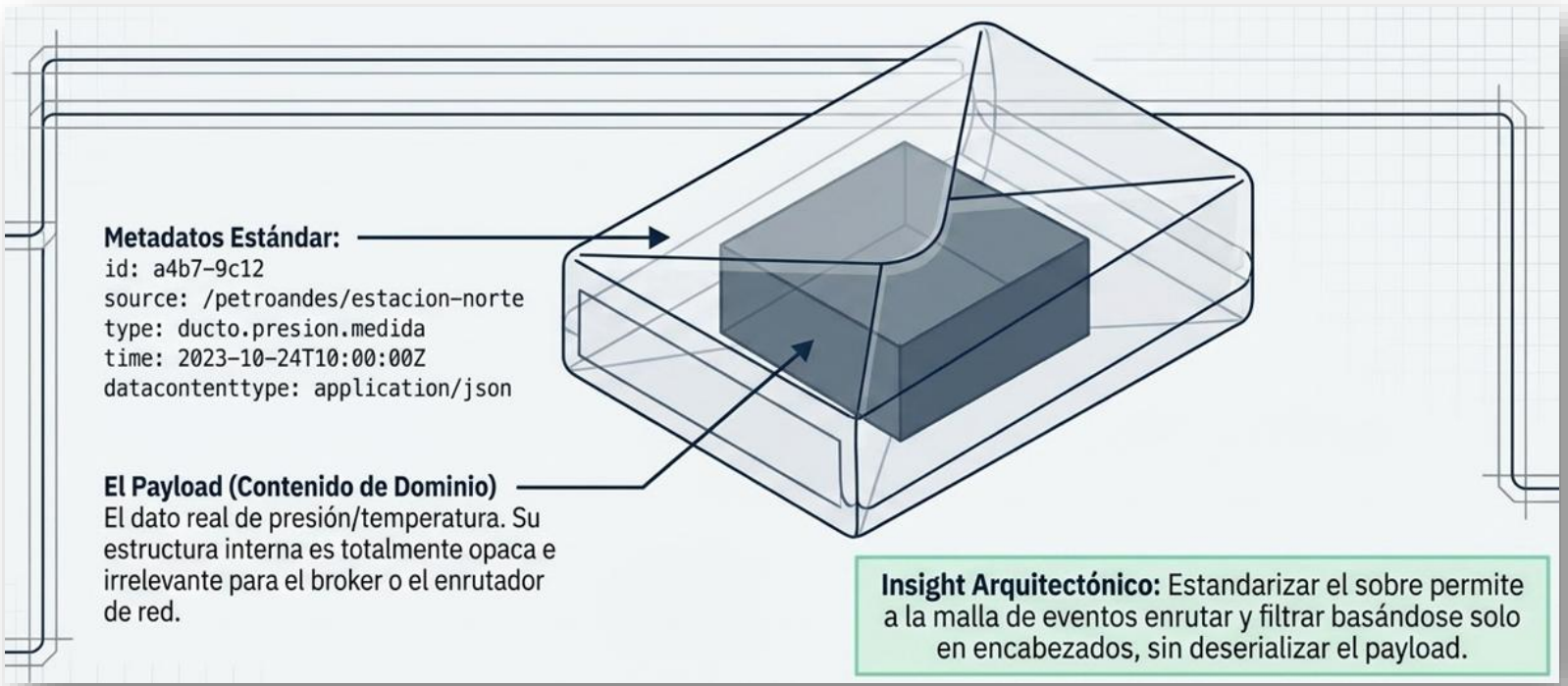
CONVENCIÓN PARA PETROANDES

type: co.petroandes.ductos.presion.medida.v1
source: /andestransporte/estacion/AYA-03
¿La versión va en el type o en el esquema? Decidirlo en el tablero.

5 Bloque 3 — CloudEvents: estandarizar el sobre

El argumento del modelo canónico, aplicado a los metadatos

OBJETIVO 2



5 Bloque 3 — AsyncAPI 3.0: el contrato del canal

18 min · contrato antes que código, también en lo asíncrono

OBJETIVO 2

ESTRUCTURA DEL DOCUMENTO

- info
- servers
- channels
- operations
- components.messages
- components.schemas

Documenta lo que OpenAPI no puede: canal, protocolo y bindings.

El cambio conceptual de la versión 3.0

Canales y operaciones están separados: la operación declara si la aplicación documentada envía (send) o recibe (receive) por ese canal. Se acabó la ambigüedad de «publish/subscribe, ¿desde el punto de vista de quién?».

ERRORES FRECUENTES

- Documentar el topic pero no el esquema del payload
- Poner comandos en canales de eventos
- Olvidar el canal de errores o dead letter

La meta del bloque siguiente no es un documento largo, sino un contrato con cero errores de linting.

5 Práctica B — Contrato telemetria-ductos

30 min · en grupos · AsyncAPI Studio o YAML plano · se sube al LMS

OBJETIVO 2

1	**Canales**	ductos.telemetria.presion · ductos.telemetria.caudal · ductos.alertas.anomalia
2	**Mensajes**	CloudEvents con payload tipado: estación, tramo, timestamp, valor, unidad y calidad del dato
3	**Operations y servers**	send/receive desde la perspectiva del servicio de telemetría; broker de desarrollo
4	**dataschema**	Declarar en el mensaje el esquema que apuntará al registry
5	**Validar**	Linter sin errores: es el criterio de cierre del bloque

CALIDAD DEL DATO

Es el campo que casi todos olvidan y sin el cual el balance no puede descartar lecturas dudosas. Si no está en el contrato, no existe para el consumidor.

Este YAML es el borrador del entregable del Taller 1 (15% de su rúbrica).

5 Bloque 5 — Schema registry y compatibilidad

25 min · control de gobierno, no optimización

OBJETIVO 3

QUÉ HACE

- Almacena y versiona los esquemas; el productor envía el identificador, no el esquema completo
- Valida al registrar, no cuando ya explotó el consumidor en producción
- Avro, Protobuf, JSON Schema

Regla	Se actualiza primero	Permite	Prohíbe
BACKWARD	los consumidores	eliminar campos; agregar opcionales con default	agregar un campo obligatorio
FORWARD	los productores	agregar campos; eliminar opcionales	eliminar un campo obligatorio
FULL	cualquiera	solo cambios compatibles en ambos sentidos	casi todo cambio estructural
NONE	—	todo	nada: por eso rompe en producción

Mnemotecnia para el oral: backward = el consumidor nuevo lee datos viejos · forward = el consumidor viejo lee datos nuevos.

Ticket de salida: ¿qué regla elegiría para el topic de telemetría y qué cambio quedaría prohibido con ella?

5 Ejercicio de ruptura — ¿permitido o prohibido?

7 min · en parejas · sobre el esquema de telemetría escrito en la práctica B · regla vigente: BACKWARD

OBJETIVO 3

1. Agregar sensorId como campo obligatorio
2. Agregar calidadDato con valor por defecto
3. Renombrar valor a valorMedido
4. Cambiar unidad de texto libre a enumeración
5. Eliminar observaciones, que nadie usa

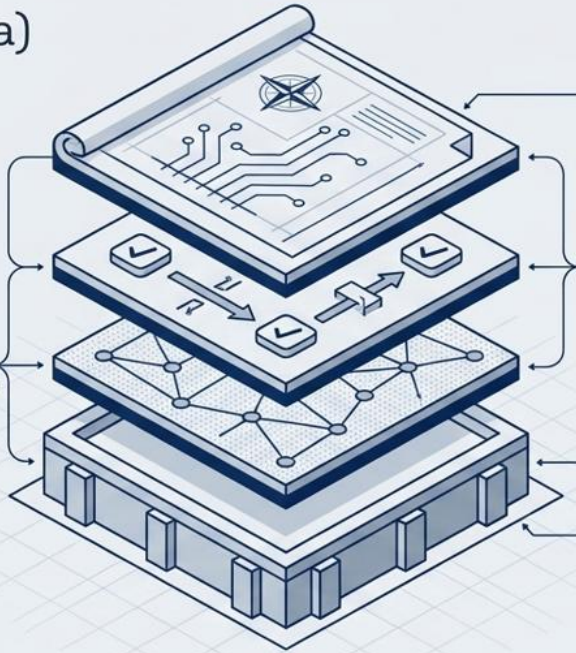
SE PIDE

Clasificar cada cambio como permitido o prohibido, y cuando esté prohibido, proponer el camino alternativo que llega al mismo resultado sin romper a los consumidores.

Pregunta clásica del Parcial 1 oral: explique cómo evolucionaría el esquema del evento sin romper a los consumidores.

5 Sintesis

El Stack EDA Completo (Síntesis Arquitectónica)



4. Gobierno & Documentación (Capa Superior)

AsyncAPI 3.0
El catálogo vivo que define contratos formales: quién publica y quién escucha en toda la red.

3. Contenido & Validación (Capa de Datos)

JSON / Avro + Schema Registry
Garantizan la calidad tipográfica y previenen la corrupción masiva de datos.

2. Enrutamiento Neutral (Capa de Red)

CloudEvents
Estandariza la metadata permitiendo mover datos sin necesidad de inspeccionarlos.

1. Transporte & Retención (Capa Base)

1. Transporte & Retención (Capa Base)

Brokers distribuidos (Kafka, Redpanda) / Event Mesh
Proveen el canal de streaming persistente y particionado.

Una arquitectura orientada a eventos madura requiere mucho más que desplegar un broker. Exige **contratos claros, estándares de enrutamiento** y **gobierno estricto** para evitar convertirse en un **monolito asíncrono** e inmanejable.

5 Cierre y laboratorio de la semana

5 min en clase · el laboratorio se realiza fuera de clase y se retoma al inicio de la sesión 6

EVIDENCIA A ENTREGAR (UNA PÁGINA O TRES CAPTURAS)

- Redpanda y su consola levantados con el docker-compose del repositorio starter
- Productor publicando lecturas de tres estaciones como CloudEvents
- Dos consumer groups (balance y alertas) leyendo el mismo topic: el fan-out sin duplicar publicación
- Captura del rechazo al registrar una versión incompatible con BACKWARD
- Una línea de conclusión: qué observó al rebobinar el offset de un consumer group

PREPARACIÓN IMPRESCINDIBLE

Instalar Docker, clonar el repositorio y ejecutar docker compose pull con buena conexión. Verificar que los puertos 8080, 8081 y 9092 estén libres.

SOPORTE

Monitoría de la semana y canal del LMS. Quien no logre levantar el entorno puede trabajar en pareja: el objetivo es observar el comportamiento.

No se califica como taller, pero es insumo directo del Taller 1 y se verifica al inicio de la sesión 6.

5 Bibliografía de la sesión

LECTURAS OBLIGATORIAS

- Bellemare, A. Building Event-Driven Microservices. O'Reilly, 2020 — caps. 1–3 (fundamentos y comunicación event-driven).
- AsyncAPI Initiative. AsyncAPI 3.0 — tutorial oficial: channels, operations, messages. asyncapi.com
- CNCF. CloudEvents Specification — atributos obligatorios del sobre. cloudevents.io

OPCIONALES

- Urquhart, J. Flow Architectures. O'Reilly, 2021 — cap. 1.
- Redpanda / Apache Kafka — documentación de conceptos: topics, particiones, consumer groups, retención.
- Confluent / Redpanda — Schema Registry: reglas de compatibilidad y evolución de esquemas.
- Kleppmann, M. Designing Data-Intensive Applications — cap. 11 (Stream Processing).